**DATA 200: Data Systems for Data Analytics**
**Project 2: Relational Model**

- This project is to be done individually with a completed Jupyter Notebook submitted on Moodle.

- In this project you will combine your understanding of how to acquire data from a relational database using SQL with your knowledge of Pandas to investigate the IMDb database.

- Launched online in 1990 and a subsidiary of Amazon.com since 1998, IMDb.com is the world's most popular and authoritative source for movie, TV and celebrity content, designed to help fans explore the world of movies and shows and decide what to watch. The database includes millions of movies, TV and entertainment programs, and cast and crew members.

- *Subsets* of the IMDb datasets are freely available, updated daily, and are contained in a set of gzipped tab-separated-values (TSV) formatted files. Using a script, these datasets were downloaded and used to construct an SQLite database file that contains all the IMDb data (that is available) as of 3/27/2022. Unfortunately, the file is 10gigs in size, so you will have to ensure that you have enough hard drive space on your computer to download the file. If you do not have enough space on your personal computer, then you could save it to OneDrive and use it from there. Alternatively, you can download the file onto one of the lab computers in Tome 121 and do your analysis there. A link to the database can be found in Teams.

- Below is a description of the tables contained within the IMDb SQLite database.

  **akas:**
  - **titleid** (string) - an alphanumeric unique identifier of the title
  - **title** (string) – the localized title
  - **region** (string) - the region for this version of the title
  - **language** (string) - the language of the title
  - **types** (array) - Enumerated set of attributes for this alternative title. One or more of the following: "alternative", "dvd", "festival", "tv", "video", "working", "original", "imdbDisplay". New values may be added in the future without warning
  - **attributes** (array) - Additional terms to describe this alternative title, not enumerated
  - **isOriginalTitle** (boolean) – 0: not original title; 1: original title

**crew:**

- **title_id** (string) - an alphanumeric unique identifier of the title
- **person_id** (string) - alphanumeric unique identifier of the name/person
- **category** (string) - the category of job that person was in
- **job** (string) - the specific job title if applicable, else '\N'
- **characters** (string) - the name of the character played if applicable, else '\N'

**episodes:**

- **episode_title_id** (string) - alphanumeric identifier of episode
- **show_title_id** (string) - alphanumeric identifier of the parent TV Series
- **season_number** (integer) – season number the episode belongs
- **episode_number** (integer) – episode number of the in the TV series

**people:**

- **person_id** (string) - alphanumeric unique identifier of the name/person
- **name** (string) – name by which the person is most often credited
- **born** (integer) – birth year in YYYY format
- **died** (integer) – year of death if applicable, otherwise NULL

**ratings:**

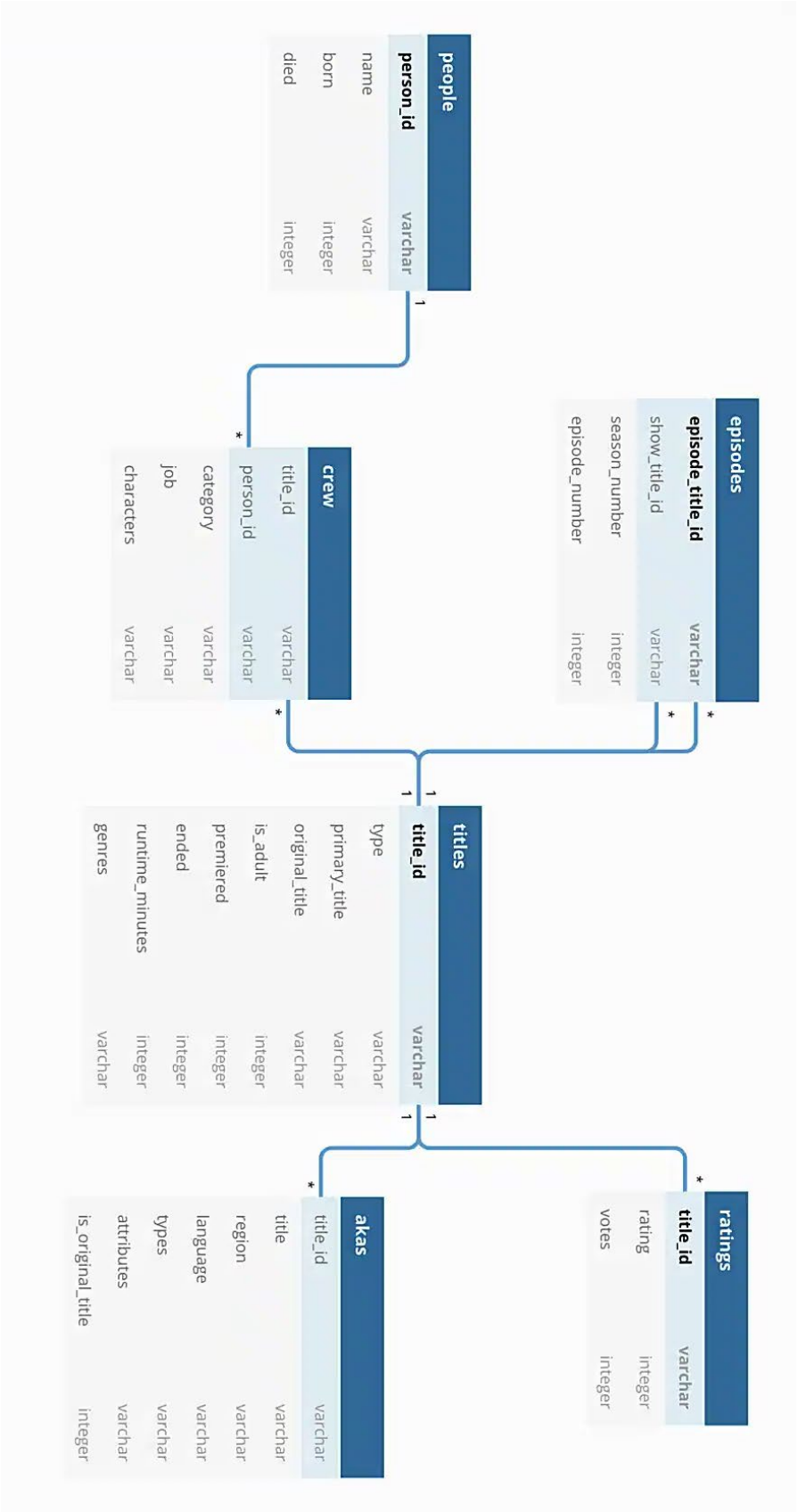
**title_id** (string) - alphanumeric unique identifier of the title
**rating** (integer) – weighted average of all the individual user ratings
**votes** (integer)- number of votes the title has received

**titles:**

- **title_id** (string) - alphanumeric unique identifier of the title
- **type** (string) – the type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc)
- **primary_title** (string) – the more popular title / the title used by the filmmakers on promotional materials at the point of release
- **original_title** (string) - original title, in the original language
- **is_adult** (boolean) - 0: non-adult title; 1: adult title
- **premiered** (YYYY) – represents the release year of a title. In the case of TV Series, it is the series start year
- **ended** (YYYY) – TV Series end year. '\N' for all other title types
- **runtime_minutes** – primary runtime of the title, in minutes
- **genres** (string array) – includes up to three genres associated with the title

- A schema of the IMBd SQLite database is provided on the next page.

# IMBd SQLite Database Schema



**Task #1:**

Write **two Python functions** that execute SQL queries of *moderate to high complexity* to provide insight into some aspects of the data contained in the IMBd database. You get to decide what these functions examine, and you are encouraged to be as creative as possible. It would be best if your functions could provide output that would be useful in some type of data analysis. However, your functions must follow these specifications:

1. The SQL queries must involve multiple tables (at least two) and should also compute columns and do group-by and/or aggregation.
2. Since this is a relational database project, the "heavy lifting" must be done using SQL, not by subsequent manipulation in Pandas. That is, you should avoid using Pandas to perform joins and grouping, rather, this should be done using SQL.
3. Your functions must have parameters (preferrable 2 or more, not including the database connection) that *abstract* your queries. That is, your queries must somehow be based on the input parameters of the function.
4. The output must have at least one visualization (but could have more), using either matplotlib or Seaborn. The visualizations must have clear axes, labels, titles, and units, so that someone who did not generate the visualization can interpret it.

**Task #2:**

Create a Jupyter notebook that clearly explains the functions from Task #1 (including comments within the code) and demonstrates them in action. You should include multiple examples of output within the notebook and comment on what you find interesting. A good notebook should allow an audience/reader who is not associated with this course to understand the development and your conclusions. **Note that this Jupyter notebook will be utilized to partially satisfy the WiD thread in the data analytics major. As such, you should write your notebook in such a way that you would be enthusiastic about showing a potential employer your project.** Please make use of section headings, bold and italicized fonts, etc.

In summary, you should put together a coherent essay in the form of a Jupyter notebook that presents the code and rationale for your functions, and examines output from the functions to point out interesting observations. (500-750 words)

**Some Thoughts:**

- Before you come up with your interesting functions, you are going to need to spend some time becoming familiar with the database, including doing some queries to explore relationships.
- The `IMDb.db` database is huge, and as such, some of your queries could take a long time (longer than your think)! Be patient and thoughtful about using joins.
- There are all sorts of visualizations you could create with either matplotlib or Seaborn, such as histograms, distribution plots, bar charts, boxplots, heat maps, violin plots, regression plots, etc.
- The database has information on movies, tv shows, etc. Keep this in mind when executing your queries and coming up with your questions (do you want to just focus on one or all types?). For example, `analyzeActor()` only focuses on movies.
- While gender is not specifically included in the data base, the words "actress" and "actor" are contained in the `crew` table.
- When looking at ratings, some only have a small number of votes. Maybe only consider ratings that have larger number of votes.
- Consider things such as age of actors, number of movies an actor starred in, differences between domestic (US) and non-domestic movies, etc. There are lots of things you could explore.
- Be careful about your query results having duplicates—this can be an issue depending on how you perform joins and the organization of the data. **I found that I needed to include the DISTINCT keyword a lot while I was exploring the data set.**
- The `genres` field (string) of the `titles` table is a jumbled mess, mixing together all different genres. You could use `LIKE '%Comedy%'` to extract comedies, for example. If you did examine genres, you may want to focus on a select few of main genres.
- Take a look at this webpage, which provides a nice example of an analysis (obviously you can't do the same one):

  IMDb 4: Querying the IMDb MySQL database and visualising its data

- The list of things you could consider is endless:
  - How many films were in a particular decade (a span of ten years)?
  - Which films have the largest casts for a particular year?
  - Which directors have directed 500 or more movies?
  - Does runtime have anything to do with ratings? For example, is there a difference between movies that have runtimes ≤ 90 minutes, runtimes between 90 and 180 minutes, and those greater than 180 minutes? You could use categorical variables for this.